

MethylPipeline Usage Manual

Operational runbook for DomainProgram workflows and staged validation

MethylPipeline Team

2026-07-07

Table of contents

1	MethylPipeline Usage Manual	7
1.1	What this manual is	7
1.2	Reading paths	7
1.3	End-to-end workflow map	7
1.4	Scope boundaries	8
1.5	Canonical references	8
2	Relationship to Theory and Implementation	9
I	Part I: Setup and Package Orientation	10
3	Prerequisites and Environment	11
3.1	Purpose	11
3.2	Required tools	11
3.3	Environment bootstrap	11
3.4	Verify CLI availability	11
3.5	Runtime assumptions	12
3.6	Success checks	12
3.7	Common failures and fixes	12
3.8	See also	12
4	Project Config and Layout	13
4.1	Purpose	13
4.2	Four-layer configuration	13
4.3	Repository vs <code>/work</code> (what goes where)	13
4.4	Minimum study manifest	14
4.5	Validation / Monte Carlo keys (profile <code>actionConfig.validation</code>)	14
4.6	Output layout	15
4.7	Handoff model	16
4.8	Do not do this	16
4.9	Success checks	16
4.10	See also	16
5	Sample Prep and Quality Control	17
5.1	Purpose	17
5.2	End-to-end flow	19
5.3	Where artifacts live	20
5.4	Alignment QC: inputs and normalized JSON	20
5.4.1	Picard-style duplication metrics	20
5.4.2	Parabricks WGBS metrics JSON	21

5.4.3	Our equivalent export JSON (V2)	21
5.5	Alignment guardrails: what “good alignment” means	22
5.5.1	Optional and analyte-specific checks	22
5.6	Cycle screening and automated recovery	23
5.7	Extraction QC: confirming enough good reads	24
5.7.1	Input: extraction manifest	24
5.7.2	Output: extraction QC JSON	24
5.7.3	Why both gates matter	25
5.8	Pass, fail, and handoff	25
5.9	Configuration checklist	26
5.10	Commands (standalone debugging)	26
5.11	Related documentation	26
6	Package Reference (Individual Usage)	27
6.1	Purpose	27
6.2	Package quick map	27
6.3	Standalone examples	28
6.3.1	Alignment QC	28
6.3.2	Enrichment from mapper output	28
6.3.3	Blind prediction with frozen model	28
6.4	Orchestrated equivalents in <code>methy1-validation</code>	28
6.5	Integration diagram	28
6.6	Do not do this	28
6.7	See also	29
II	Part II: Staged Solution Workflow	30
7	Stage: Stability	31
7.1	Purpose	31
7.2	Entry command	31
7.3	Required config keys	31
7.4	Statistical modeling modes (generic profiles)	32
7.5	Process model	32
7.6	Expected outputs	33
7.7	Success checks	33
7.8	Do not do this	33
7.9	Recovery	33
7.10	See also	34
8	Stage: Freeze	35
8.1	Purpose	35
8.2	Entry command	35
8.3	Prerequisites	35
8.4	Process model	35
8.5	Expected outputs	35
8.6	Success checks	36
8.7	Recommended post-freeze gate	36

8.8	Do not do this	36
8.9	Recovery	36
8.10	See also	36
9	Stage: Model	37
9.1	Purpose	37
9.2	Common execution patterns	37
9.2.1	Pattern A: Direct production model build	37
9.2.2	Pattern B: Backend model-MC comparison then select best	37
9.3	Backend behavior	37
9.4	Process model	38
9.5	Required config keys	38
9.6	Expected outputs	38
9.7	Success checks	39
9.8	Do not do this	39
9.9	See also	39
10	Stage: Post-model Validation	40
10.1	Purpose	40
10.2	Entry command	40
10.3	Prerequisites	40
10.4	Process model	40
10.5	Expected outputs	40
10.6	Success checks	41
10.7	Do not do this	41
10.8	Recovery	41
10.9	See also	41
11	Stage: Blind Prediction	42
11.1	Purpose	42
11.2	Important boundary	42
11.3	Entry command (standalone)	42
11.4	Process model	42
11.5	Required config pattern	42
11.6	Expected outputs	42
11.7	Success checks	43
11.8	Do not do this	43
11.9	Recovery	43
11.10	See also	43
III	Part III: Operations and Reliability	44
12	Artifacts and QA Checks	45
12.1	Purpose	45
12.2	Workflow action observability	45
12.3	Artifact map	45
12.4	Handoff integrity checks	46

12.5	QA checklist per stage	46
12.5.1	Stability	46
12.5.2	Freeze	46
12.5.3	Model	46
12.5.4	Post-model validation	47
12.5.5	Blind prediction	47
12.6	Escalation triggers	47
12.7	See also	47
13	Troubleshooting and Recovery	48
13.1	Purpose	48
13.2	Failure routing	48
13.3	Common problems	48
13.4	Safe recovery sequence	49
13.5	Resume examples	49
13.6	Operational guardrails	49
13.7	See also	49
14	Command Cookbook	50
14.1	Purpose	50
14.2	1) Full staged run (strict profile example)	50
14.3	2) Backend comparison and selection	50
14.4	3) Rollout comparison report	50
14.5	4) Standalone enricher run from mapper output	51
14.6	5) Standalone blind prediction	51
14.7	6) Manual rendering commands (this manual)	51
14.8	7) Distributed queue (one task)	51
14.9	8) Hyperparameter search grid (example)	51
14.10	Execution notes	52
15	Distributed MethyValidation on shared storage	53
15.1	Purpose	53
15.2	Prerequisites	53
15.3	Layout (under <code>monte_carlo_runs/</code>)	53
15.4	Command sequence	54
15.5	Example paths (<code>/work</code> pattern)	54
15.6	Recovery	54
15.7	See also	55
16	Deployment and Distributed Workflow	56
16.1	Purpose	56
16.2	Architecture	56
16.3	Greenfield checklist	57
16.4	Database deploy (dual-backend)	57
16.4.1	PostgreSQL (recommended for new deployments)	57
16.4.2	Azure SQL (legacy / phase-1)	58
16.5	Action catalog and workflow definitions	58
16.6	Gateway configuration	59

16.7	Worker configuration	59
16.8	End-to-end operator flow	60
16.8.1	1. SamplePrepPipeline	60
16.8.2	2. Study validation / MC	60
16.8.3	3. Local workflow (no gateway)	61
16.9	Related documentation	61
17	Optional hyperparameter search	62
17.1	When to use	62
17.2	What it is	62
17.3	Command example	62
17.4	Pitfalls	63
17.5	See also	63
IV	Part IV: Worked Tutorials	64
18	Tutorial: Healthy vs. Cancer (single group or increasing stages)	65
18.1	What you will build	65
18.2	Mental model (four layers)	65
18.3	Step 1 — Lay out cohort CSVs	65
18.4	Step 2 — Write the study manifest	66
18.4.1	Option A — Binary (healthy vs. one cancer group)	66
18.4.2	Option B — Increasing stages (healthy vs. stage 1 ... stage N)	66
18.5	Step 3 — Understand what <code>comparisons</code> does	67
18.6	Step 4 — Pick a profile	68
18.7	Step 5 — Run it	68
18.8	Step 6 — Find the outputs	69
18.9	Recap	70
18.10	See also	70

1 MethylPipeline Usage Manual

1.1 What this manual is

This is the operational companion to the theory book. It is designed for execution teams who need:

- exact commands,
- required config keys,
- expected output artifacts,
- stage handoff checks,
- recovery actions when a run fails.

For statistical background, use [docs/theory/](#). For compiler and worker internals, use [docs/implementation/](#). Start with [Relationship to Theory and Implementation](#).

1.2 Reading paths

- **Sample prep (FASTQ → HDF5):** Chapter 03 (`03-sample-prep-and-qc.qmd`).
- **Fast path (stage execution):** Chapters 05 to 09 (stability through blind prediction).
- **New project setup:** Chapters 01, 02, 03, then 05.
- **Guided tutorial (healthy vs. cancer stages):** Chapter 16 — end-to-end walkthrough for a single-group or multi-stage comparison.
- **Daily operations:** Chapters 10 to 12.
- **Scaling MC across machines:** Chapter 13 — shared `/work` storage and queue subcommands.
- **Deployment (DB + gateway + workers):** Chapter 14.
- **Optional hyperparameter tuning:** Chapter 15 — `methyl-hyperparam-search` grid driver.
- **Standalone package usage:** Chapter 04.

1.3 End-to-end workflow map

```
flowchart LR
  setup[Setup]
  stability[Stability]
  freeze[Freeze]
  model[Model]
  postmodel[Post-model Validation]
```

```
blind[Blind Prediction]
setup --> stability --> freeze --> model --> postmodel
model --> blind
```

1.4 Scope boundaries

- **SamplePrepPipeline** (FASTQ → aligned BAM → methylation HDF5) runs before staged validation; see Chapter 3.
- **Canonical orchestration:** `methyl-workflow-run` with repo DomainPrograms; legacy `methyl-validation --stability/--freeze/--model` documented where still used.
- Blind-only prediction is documented as a standalone `methyl-predictor` operation.
- Package behavior is documented from implementation in `packages/*`.

1.5 Canonical references

- Sample prep workflow: [workflow_engine/sql_mssql/SamplePrepFlow.md](#)
- DomainProgram language: [docs/reference/domain-program-language.md](#)
- Pipeline stages diagram: [docs/architecture/pipeline-stages.md](#)
- Validation orchestration: `packages/methylvalidation/docs/USAGE.md`
- Distributed queue: `packages/methylvalidation/docs/DISTRIBUTED_QUEUE.md`
- Hyperparameter search: `packages/methylvalidation/docs/HYPERPARAMETER_SEARCH.md`

2 Relationship to Theory and Implementation

This usage manual owns **operator commands**, stage handoffs, artifact gates, and deployment checklists. It does not derive statistical methods or document compiler internals.

Need	Read
DMP detection math, assumptions, limitations	Theory book — start with ch.03
DomainProgram language and schemas	Reference: domain-program-language
Parameter keys and defaults	Reference: configuration-reference
Workflow engine, workers, compiler	Implementation guide
System layers and orchestration paths	Architecture

Canonical orchestration: `methyl-workflow-run` with repo DomainPrograms and `/work` study manifests. Legacy `methyl-validation --stability/--freeze/--model` remains documented where operators still use it; prefer DomainProgram paths for new studies.

Repo vs /work: profiles and programs live in the git repo; `project_*.json` and run artifacts live under `/work/<disease>/`. See ch.02 and [work-config-paths rule](#).

Part I

Part I: Setup and Package Orientation

3 Prerequisites and Environment

3.1 Purpose

Set up a reproducible execution environment for all package CLIs and workflow stages.

3.2 Required tools

- Python environment in repo-local `.venv`
- Installed package CLIs (`methyl-*`)
- Quarto for manual rendering (optional for pipeline runs)
- TeX engine for PDF rendering (`quarto install tinytex` recommended)

3.3 Environment bootstrap

From repo root:

```
python3.12 -m venv .venv
source .venv/bin/activate
bash scripts/install_all.sh
```

Optional dependency sets:

```
bash scripts/install_all.sh --pipeline-reqs
bash scripts/install_all.sh --gpu-reqs
```

3.4 Verify CLI availability

```
source .venv/bin/activate
methyl-validation --help
methyl-centroid --help
methyl-detector --help
methyl-mapper --help
methyl-enricher --help
methyl-predictor --help
```

3.5 Runtime assumptions

- Activate `.venv` before every run.
- Use absolute paths in project configs where possible.
- Keep `samples_base_path` consistent across stages.

3.6 Success checks

- `.venv` activates without import errors.
- `methyl-validation --help` prints options.
- Project JSON path resolves on the execution host.

3.7 Common failures and fixes

- **command not found:** activate `.venv` and reinstall with `scripts/install_all.sh`.
- **module import errors:** reinstall editable packages from repo root.
- **path mismatch between hosts:** set profile `actionConfig.validation.path_remap` or `--path-remap OLD=NEW`.

3.8 See also

- `README.md`
- `docs/DEPLOYMENT.md`

4 Project Config and Layout

4.1 Purpose

Define a **study manifest** (slim project JSON) plus reusable **profile**, **site**, and **program** artifacts so stability, freeze, model, and evaluation run without per-stage schema drift.

4.2 Four-layer configuration

Layer	Typical path	Owns
Study manifest	<code>/work/<disease>/configs/project_*.json</code>	Cohorts, stages, comparisons, paths, regulatory, validation_partitions, progression_order
Pipeline profile	<code>workflow_engine/domain/profiles/*.profile.json</code>	actionConfig tool parameters + scope booleans (runDmpSelection, ...)
Site manifest	<code>/work/site/methyl_site.json</code> (or <code>METHYL_SITE_CONFIG</code>)	Genomes, GTF, caches, cluster defaults
DomainProgram	repo <code>workflow_engine/domain/**/*.program.json</code>	Control flow, per-action with / stepOverride

Precedence (highest wins): program/instance override → profile actionConfig → analyte defaults (regulatory.primary_analyte) → site manifest → *(no Python fallback for tunable science knobs)*.

Study manifests **must not** contain `step_config`. Legacy projects: `python scripts/migrate_project_config.py --in-place path/to/project.json`.

4.3 Repository vs /work (what goes where)

Repository (versioned, documented)	/work/<disease>/ (study-specific)
Pipeline profiles (workflow_engine/domain/profiles/)	project_*.json — cohorts, comparisons, sample paths, output_base, regulatory
DomainPrograms (*.program.json under workflow_engine/domain/)	Sample list CSVs under data/
JSON schemas (schemas/config/)	Run outputs (monte_carlo_runs/ stability/, ...)
Site example (tools/methyl-config- editor/configs/site_grch38.example.json)	/work/site/methyl_site.json (operator copy)

- Point methyl-workflow-run --program at a **repo** DomainProgram path.
- Point --context / projectPath at a **study manifest on /work**.
- Pass --context-file with a profile from workflow_engine/domain/profiles/, or set METHYL_PROFILE.
- Set METHYL_SITE_CONFIG (or install site manifest under /work/site/).
- Do **not** treat repo workflow_engine/domain/checks/*/configs/project_*.json as production configs (CI smoke only).
- Do **not** copy *.program.json into /work/<disease>/configs/.

4.4 Minimum study manifest

Use one slim project file containing:

- global cohort metadata (controls, diseases, comparisons, optional stages[] / progression_order),
- filesystem anchors (output_base, samples_base_path),
- optional regulatory, validation_partitions, chromosomes, contexts.

Tool parameters live in the **profile** actionConfig sections (detection, validation, mapper, ...), not in the study manifest.

4.5 Validation / Monte Carlo keys (profile actionConfig.validation)

Resolved into MonteCarloConfig by methyl-validation --project (via METHYL_PROFILE):

- split policy: train_fraction, n_iterations, seed
- stability controls:
 - run_stability
 - stability_dmp_freq
 - stability_min_balanced_accuracy
 - stability_gene_freq
 - stability_featurecuts_enabled
 - dmp_featurecuts_target_ba, gene_featurecuts_target_ba (split BA gates)
 - dmp_modeling_mode, gene_modeling_mode (statistical axes)

- `stability_gene_featurecuts_max_dmps`, `stability_gene_featurecuts_max_genes` (site/profile caps)
- freeze/model paths:
 - `freeze_stable_dmp_csv` (optional)
 - `production_output_dir` (optional)
- backend/model:
 - `model_backend`
 - `feature_mode`
 - `ecdf_second_stage_enabled`
 - backend-specific `tabular_*` / `generative_*` keys

Sample prep uses profile `actionConfig.alignment_qc` and `actionConfig.extraction_qc` ([Chapter 3](#)).

4.6 Output layout

Most runs write to:

`<output_base>/<project_name>/monte_carlo_runs/`

Important subtrees:

- `run_XXXX/` (per-iteration artifacts)
- `stability/` (stable panel and summaries)
- `production/` (freeze and final model outputs)
- `model_mc/` (backend comparison runs)
- `post_model_validation/` (frozen-model descriptive evaluation)

4.7 Handoff model

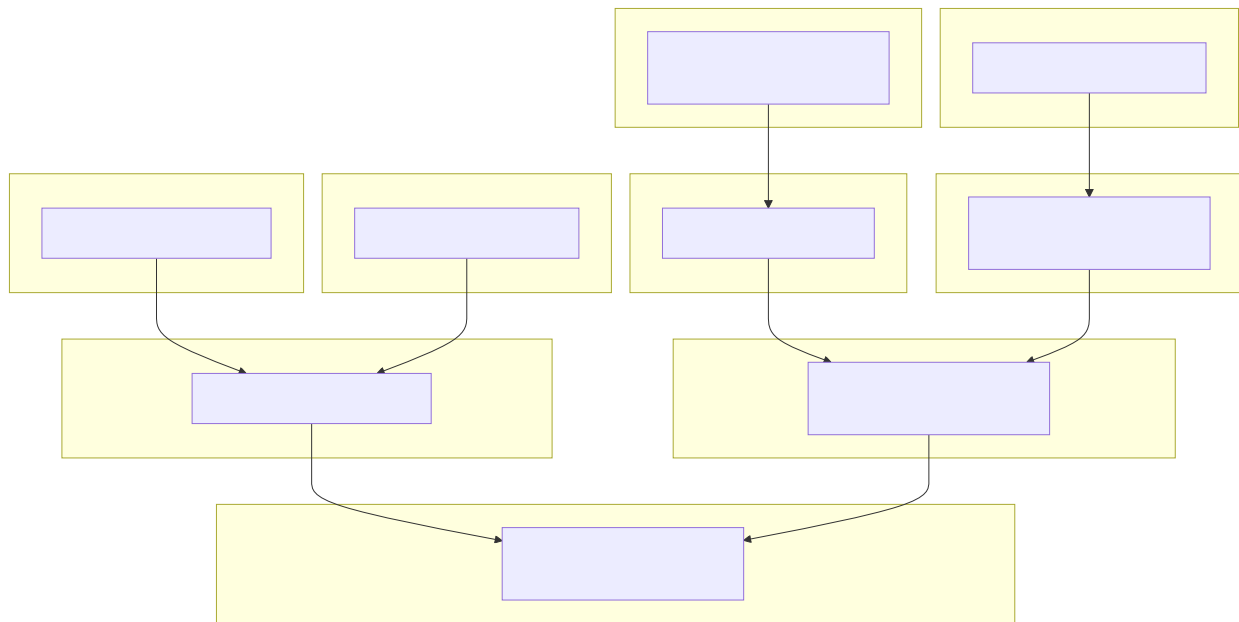


Figure 4.1: Configuration layer model

4.8 Do not do this

- Do not embed tool parameters in the study manifest (`step_config` is rejected at load).
- Do not put blind-only predictor config into validation-driven runs.
- Do not change `samples_base_path` between stages without explicit remap.
- Do not run `--freeze` before stable panel exists.

4.9 Success checks

- Study manifest loads via `methy1-utils / methy1-validation --project ....`
- `METHYL_PROFILE` resolves `actionConfig.validation` (or pass `--config mc.json`).
- Run directories appear under expected root.

4.10 See also

- `docs/reference/config-parameter-matrix.md`
- `docs/reference/domain-program-language.md`
- `docs/theory/chapters/11-project-configuration.qmd`
- `docs/reference/configuration-reference.qmd`

5 Sample Prep and Quality Control

5.1 Purpose

SamplePrepPipeline is the per-sample upstream workflow that turns raw WGBS FASTQs into methylation HDF5 files (`{chromosome}-CG.h5`) that downstream stages consume. Every sample passes through **two automated guardrails** before it enters stability, freeze, or discovery workflows:

1. **Alignment QC** (`methyl-qc`) — confirms Parabricks alignment and sequencing metrics are acceptable (or fixable via trim/realign).
2. **Extraction QC** (`methyl-extraction-qc`) — confirms MethylExtractor produced sufficient coverage and conversion-quality signal across chromosomes.

Without these gates, bad alignments would waste GPU extraction time, and under-covered or poorly converted extractions would pollute centroids, DMP discovery, and classifiers.

Workflow source of truth: [workflow_engine/domain/fixtures/sample_prep.program.json](#)

Operator reference: [workflow_engine/sql_mssql/SamplePrepFlow.md](#)

5.2 End-to-end flow

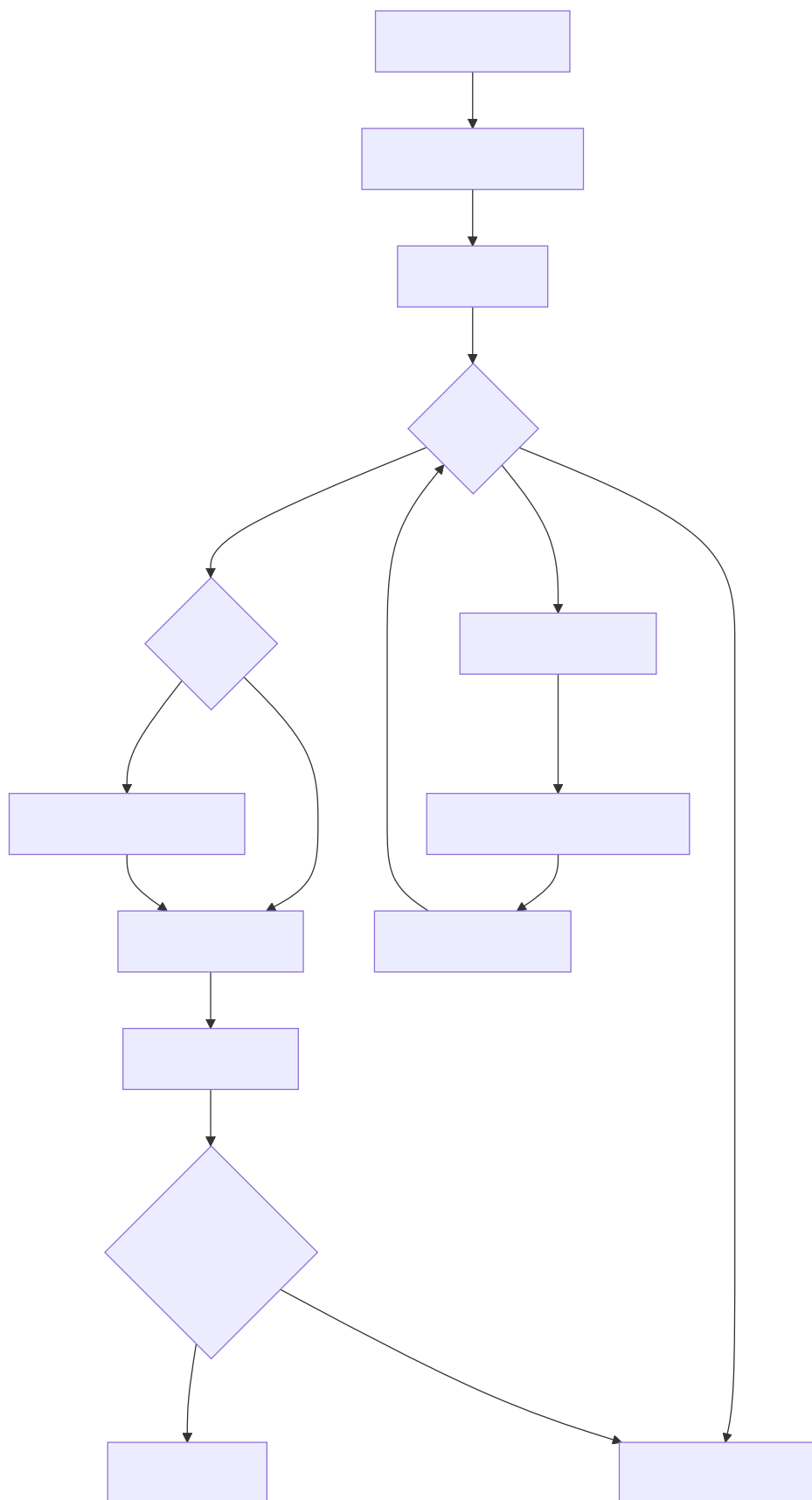


Figure 5.1: Sample¹⁹ prep and QC flow

Gate	Scope variable	When	Blocks
Alignment	<code>qcPass</code>	After Parabricks, before extract	Extraction (unless remediation succeeds)
Extraction	<code>extractionQcPass</code>	After MethylExtractor	Full archive and BAM deletion

Ordering rationale: alignment QC runs before cfDNA fragmentomics so failed samples skip BAM scanning. Fragmentomics and extraction both need the aligned BAM. FASTQs are retained until QC is fully resolved so **fastp** can trim and Parabricks can realign on the remediation path.

5.3 Where artifacts live

Per sample under `/work/samples/{sample_id}/`:

Artifact	Role
<code>*deduplicate_metrics.txt</code>	Picard-style duplication metrics from Parabricks
<code>{sample_id}.json</code>	Parabricks WGBS metrics JSON (sequencing + alignment)
<code>{sample_id}.qc-metrics.tar</code>	Optional tar of tabular metrics (used when JSON is absent)
<code>{output_base}/{project}/alignment_qc/{sample_id}.json</code>	Normalized V2 alignment QC export (when run project-scoped)
<code>{sample_id}.extraction_manifest.json</code>	MethylExtractor summary (written at extract time)
<code>{sample_id}.extraction_qc.json</code>	Extraction guardrail report
<code>{sample_id}.sample_prep_log.jsonl</code>	Append-only audit of every prep action
<code>{chr}-CG.h5</code>	Methylation matrices consumed by <code>methyl-centroid</code>

5.4 Alignment QC: inputs and normalized JSON

`methyl-qc` (`packages/methylalignmentqc`) combines **two upstream metric families** into one decision:

5.4.1 Picard-style duplication metrics

Parabricks emits a tab-separated **deduplication metrics** file (names vary: `*deduplicate_metrics.txt` or `*duplication_metrics.txt`). The parser (`methyl_alignment_qc/core/parser.py`) reads Picard-compatible columns such as:

- `LIBRARY`, `UNPAIRED_READS_EXAMINED`, `READ_PAIRS_EXAMINED`
- `SECONDARY_OR_SUPPLEMENTARY_RDS`, `UNMAPPED_READS`
- `UNPAIRED_READ_DUPLICATES`, `READ_PAIR_DUPLICATES`
- `PERCENT_DUPLICATION`

These feed `summary_stats` on the export JSON (total reads, duplication rate, etc.). Optional config guardrails (`actionConfig.alignment_qc.optional_guardrails.duplication_rate_max`) can fail samples with excessive PCR duplication even when cycle quality looks fine.

5.4.2 Parabricks WGBS metrics JSON

Each sample directory should contain `{sample_id}.json`: Parabricks' structured report with sections such as:

- `quality_yield` — total/PF reads and Q30 bases
- `mean_quality_by_cycle` — per-cycle mean Phred (Read 1 then Read 2; for 151+151 cycles, Read 2 starts at cycle 152)
- `gc_bias_summary`, `insert_size_metrics`
- `pre_adapter_summaries` — deamination and OxoG artifact qscores

When this file is present, guardrails in `wgbs_parabricks_qc.py` evaluate sequencing and alignment health **before** methylation extraction.

5.4.3 Our equivalent export JSON (V2)

The worker writes a normalized **V2 row-oriented** JSON per sample (schema: `schemas/config/alignment_qc/export`). Key blocks:

Block	Contents
<code>summary_stats</code>	Duplication rate, read counts from Picard metrics
<code>quality_yield</code> , <code>mean_quality_by_cycle</code> , ... <code>guardrails.details</code>	Parabricks sections (embedded or referenced) Per-metric pass/fail, observed value, normal range, operator message
<code>guardrails.screening</code> <code>guardrails.overall_pass</code> <code>qc_history</code>	Cycle-quality disposition and recommended trim counts Single boolean bound to workflow <code>qcPass</code> Append-only list of evaluations (initial + post-remediation retries)
<code>fragmentomics_metrics</code>	cfDNA insert-size guardrails when enabled

V1-shaped payloads are assembled internally and converted to V2 for export; standalone tools can migrate older files with `methy1-qc-migrate-guardrails`.

5.5 Alignment guardrails: what “good alignment” means

Core WGBS guardrails (defaults in `wgbs_parabricks_qc.py`) include:

Metric	Default expectation	Why it matters
PF read percent	90%	Run-level filtering quality
Q30 percent	85% (configurable)	Base-call confidence
Mean quality	35	Overall sequencing noise
Min quality after cycle 20	30	Late-cycle degradation
AT / GC dropout	< 3 / < 5	Coverage bias across genome
Median insert size	150–300 bp	Library fragmentation sanity
Deamination qscore	30	Expected WGBS conversion signal
OxoG qscore	20	Oxidative damage risk

Each metric carries a **message** explaining the biological/operational concern. `guardrails.overall_pass` is the logical AND of all evaluated checks (including optional and analyte-specific add-ons).

5.5.1 Optional and analyte-specific checks

Configure under profile or site `actionConfig.alignment_qc` (not in the study manifest):

```
{
  "alignment_qc": {
    "optional_guardrails": {
      "duplication_rate_max": 0.25,
      "min_pf_reads": 1000000
    },
    "bisulfite_conversion": {
      "enabled": true,
      "source": "auto",
      "min_conversion_rate_pct": 99.0
    },
    "fragmentomics": {
      "enabled": true,
      "profile": "cfdna"
    },
    "cycle_screening": {
      "enabled": true,
      "r2_quality_threshold": 30,
      "max_trim_bases": 8
    },
    "alignment_guardrails": {
      "enabled": true,
      "min_mapping_rate": 0.98,
      "flagstat_enabled": true,

```

```

    "min_properly_paired_rate": 0.90
  }
}

```

- **Alignment guardrails:** mapping rate, secondary/supplementary rate, GC uniformity (Picard-derived), and properly paired rate (`samtools flagstat` on the BAM). Enabled by default for `cfDNA` and `buffy_coat` via analyte profile. See [implementation guide](#).
- **Bisulfite conversion:** reads `bisulfite_conversion.json` sidecar when present; otherwise may use deamination qscore as a qualitative proxy.
- **cfDNA fragmentomics:** when `primary_analyte` is `cfDNA`, insert-size histogram guardrails can flag abnormal fragment profiles before extraction.

See [docs/ANALYTE_PROFILES.md](#) for profile defaults.

5.6 Cycle screening and automated recovery

Historically, a single `min(mq[20:])` over the combined cycle series could fail fixable Read-2 start dips with a generic “do not proceed” message. **Cycle screening** (`cycle_quality_screening.py`) inspects Read 1 and Read 2 separately and assigns a **disposition**:

Disposition	Meaning	SamplePrep action
USE_CURRENT_ALIGNMENT	Guardrails pass	Continue to extract
REALIGN_TRIM / REALIGN_READ2_TRIM	Fixable read-end quality dip	<code>sample.trim_fastq</code> → Parabricks <code>forceRealign</code> → methyl-qc retry
INVESTIGATE_MULTI_REGION	Multiple low-quality regions	Manual review; no auto-trim
INVESTIGATE_GUARDRAIL_ONLY	Cycles OK; duplica- tion/depth/insert fail	Manual review
NOT_FIXABLE	No automated path	Archive QC-only artifacts, mark failed

Screening writes `guardrails.screening` with recommended trim bases (`trim_front1`, `trim_tail1`, `trim_front2`, `trim_tail2`). SamplePrep binds `remediateAlignment` when disposition is trim-eligible and FASTQs are still on disk.

Important: FASTQs must **not** be deleted before final alignment QC disposition. The workflow deletes FASTQs only after pass or final reject (including post-remediation retry).

Offline cohort review (no re-align):

```

source .venv/bin/activate
python scripts/alignment_qc_cohort_screening.py \
  --qc-dir /work/projects/prostate-cancer/alignment_qc \
  --group healthy=/work/projects/prostate-cancer/data/healthy.csv \

```

```
--group pca=/work/projects/prostate-cancer/data/pca.csv \
--out /work/projects/prostate-cancer/alignment_qc/screening_report
```

5.7 Extraction QC: confirming enough good reads

Alignment QC validates the **BAM**. Extraction QC validates the **methylation call layer** MethylExtractor produced.

5.7.1 Input: extraction manifest

MethylExtractor writes `{sample_id}.extraction_manifest.json` with:

- `summary` — genome-wide CpG weighted mean coverage, CHH/CHG methylation levels
- `per_chromosome` — per-chromosome blocks with `CG.mean_coverage` and related stats

5.7.2 Output: extraction QC JSON

`methyl-extraction-qc` reads the manifest and writes `{sample_id}.extraction_qc.json`. Default guardrails (`methylextractionqc/guardrails.py`):

Guardrail	Default	Interpretation
<code>cpg_weighted_mean_coverage</code>	10×	Enough reads at CpG sites passing minimum coverage
<code>chh_methylation_level</code>	0.02	Low non-CpG methylation (conversion sanity) when CHH extracted
<code>chg_methylation_level</code>	0.02	Same for CHG
<code>chromosome_completeness</code>	all expected chromosomes	Every project chromosome present with CG metrics
<code>chromosome_uniformity</code>	autosomal min/median 0.5	No severe localized dropout

`guardrails.overall_pass` maps to workflow `extractionQcPass`. Failures are **terminal** today (no automatic retry loop).

Override thresholds via profile `actionConfig.extraction_qc`. Schema: `schemas/extraction_qc_output.schema`

Question	Answered by
----------	-------------

5.7.3 Why both gates matter

Question	Answered by
Did sequencing and alignment produce a usable BAM?	Alignment QC
Did we extract enough confident methylation signal genome-wide?	Extraction QC

A sample can pass alignment but fail extraction (e.g., coverage collapse on certain chromosomes). Conversely, remediation after alignment QC prevents spending extract GPU time on BAMs that will never support stable downstream statistics.

5.8 Pass, fail, and handoff

Outcome	Alignment QC	Extraction QC	Next step
Pass both	<code>qcPass: true</code>	<code>extractionQcPass: true</code>	Optional full archive → delete FASTQs/BAM → sample ready for DataDrivenPipeline
Fail alignment (not fixable)	<code>qcPass: false</code>	—	QC-only archive, mark failed, skip extract
Fail after remediation	<code>qcPass: false</code> (attempt 2)	—	Same as above
Pass alignment, fail extraction	<code>qcPass: true</code>	<code>extractionQcPass: false</code>	QC-only archive, mark failed; H5 may exist but must not enter production cohorts

When SamplePrep instance **COMPLETED**, start stability or discovery workflows with the same `projectPath`. Downstream code resolves HDF5 via `methylation_domain.helpers.resolve_methylation_h5_path()` — do not hard-code sample paths.

Workers tag scope with **MethylSampleRef** including `alignmentQc`, `extractionQc`, and `methylation` refs for traceability.

5.9 Configuration checklist

Before starting SamplePrep, confirm:

- ☐ `project.json` lists samples and `samples_base_path / path_remap` resolve to `/work/samples/...`
- ☐ Profile/site `actionConfig.alignment_qc` thresholds match analyte (cfDNA vs buffy coat)
- ☐ Profile `actionConfig.extraction_qc` min coverage appropriate for WGBS depth expectations
- ☐ `validation.regulatory.primary_analyte` set (drives fragmentomics profile)
- ☐ Instance `context_json` includes `fastqStorage`, `referenceFasta`, `samples[]`

5.10 Commands (standalone debugging)

```
source .venv/bin/activate

# Alignment QC for one sample directory
methyl-qc --samples /work/samples/SAMPLE_ID --output-dir /work/projects/prostate-cancer/alignm

# Project-scoped (uses profile actionConfig.alignment_qc via METHYL_PROFILE)
methyl-qc --project /work/projects/prostate-cancer/configs/project_Example.json

# Extraction QC after MethylExtractor
methyl-extraction-qc --sample-dir /work/samples/SAMPLE_ID --sample-id SAMPLE_ID
```

5.11 Related documentation

- **Implementation guide (guardrail boundaries, remediation, extraction filtering, cfDNA fragmentomics):** [docs/implementation/sample-preparation-flow.md](#)
- Workflow operator guide: [workflow_engine/sql_mssql/SamplePrepFlow.md](#)
- Architecture §0: [workflow_engine/docs/pipeline_architecture.md](#)
- Worker contract: [workflow_engine/contract/sample_prep_capabilities.md](#)
- Alignment QC package: [packages/methylalignmentqc/docs/USAGE.md](#)
- Extraction QC package: [packages/methylextractionqc/README.md](#)
- Historical design notes (cycle screening): [docs/plans/alignment-qc-screening.plan.md](#)

6 Package Reference (Individual Usage)

6.1 Purpose

Summarize each active package as a standalone tool, then map it to staged `methylation-validation` orchestration.

For the full SamplePrep story (Picard metrics, Parabricks JSON, guardrails, remediation, extraction QC), see [Chapter 3 — Sample Prep and Quality Control](#).

6.2 Package quick map

Package	CLI	Primary role	Typical standalone output
<code>methylationalignmentqc</code>	<code>methylation-qc</code>	Parse alignment/dup metrics into normalized JSON	<code>alignment_qc/<sample>.json</code>
<code>methylationcentroid</code>	<code>methylation-centroid</code>	Build cohort centroids from methylation files	centroid artifacts per group
<code>methylationdetector</code>	<code>methylation-detector</code>	Discover/classify DMPs	<code>dmps-*-discovery.csv</code> , <code>result*.json</code>
<code>methylationmapper</code>	<code>methylation-mapper</code>	Map DMP loci to genes/features	<code>all-gene_name-combined.csv</code>
<code>methylationenricher</code>	<code>methylation-enricher</code>	Enrichment/module interpretation	<code>enrichment_merged.csv</code> , <code>modules_ranked.csv</code>
<code>methylationclassifier</code>	<code>methylation-classifier</code>	Train production classifier (ECDF path)	<code>classifiers/*.pkl</code>
<code>methylationpredictor</code>	<code>methylation-predictor</code>	Predict and report metrics/inference	<code>predictors/*/validation_metrics</code> or blind reports
<code>methylationvalidation</code>	<code>methylation-validation</code>	Orchestrate MC/stability/freeze/model workflows	<code>monte_carlo_runs/*</code>
<code>methylationdiseaseprogression</code>	<code>methylation-disease-progression</code>	Cross-stage synthesis from mapper/enricher outputs	<code>progression/*.csv</code> , <code>summary.json</code>

6.3 Standalone examples

6.3.1 Alignment QC

```
source .venv/bin/activate
methyl-qc --metrics-root /data/metrics --output-dir /work/alignment_qc
```

6.3.2 Enrichment from mapper output

```
source .venv/bin/activate
methyl-enricher --input /work/project/mapper/pca_pca1/all-gene_name-combined.csv --modules --o
```

6.3.3 Blind prediction with frozen model

```
source .venv/bin/activate
methyl-predictor --project /work/project/monte_carlo_runs/production/project.json
```

6.4 Orchestrated equivalents in methyl-validation

- Stability stage runs centroid+detector per iteration.
- Freeze stage runs centroid->detector->mapper->enricher (and optional progression).
- Model stage runs:
 - ecdf: classifier->predictor
 - tabular_sklearn / generative_hybrid: in-process backend train/predict
- Post-model validation runs frozen-model descriptive holdouts.

6.5 Integration diagram



Figure 6.1: Package integration

6.6 Do not do this

- Do not assume standalone package defaults match project-orchestrated overrides.
- Do not skip `project.json` artifact handoff checks when chaining packages manually.

6.7 See also

- `README.md`
- `packages/*/docs/USAGE.md`

Part II

Part II: Staged Solution Workflow

7 Stage: Stability

7.1 Purpose

Run repeated train/validation iterations to identify DMPs that recur across runs and materialize a stable production panel.

7.2 Entry command

```
source .venv/bin/activate
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-C
```

Optional strict FeatureCuts alignment:

```
source .venv/bin/activate
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-C
--stability --stability-featurecuts --stability-target-ba 0.95 --stability-min-selected-dmps
```

7.3 Required config keys

In the pipeline profile `actionConfig.validation` (set `METHYL_PROFILE` or pass `--context-file`):

- `n_iterations`, `train_fraction`, `seed`
- `run_stability`
- `stability_dmp_freq`
- `stability_min_balanced_accuracy`
- `stability_gene_freq`
- optional adaptive stop controls:
 - `stability_early_stop_enabled`
 - `stability_min_iterations`
 - `stability_convergence_window`
 - `stability_convergence_jaccard`
 - `stability_convergence_max_size_delta`
 - `stability_convergence_patience`
- optional FeatureCuts controls:

- `stability_featurecuts_enabled`
- `dmp_featurecuts_target_ba`, `gene_featurecuts_target_ba` (split BA gates)
- `dmp_modeling_mode`, `gene_modeling_mode`
- `stability_gene_featurecuts_max_dmps`, `stability_gene_featurecuts_max_genes` (site/profile caps)

7.4 Statistical modeling modes (generic profiles)

Profiles are **process-agnostic**; study facts stay in the study manifest. See `docs/reference/domain-program-language.md` for the full matrix.

Goal	Profile	Key modes
DMP recurrence only	<code>mc_dmp</code>	<code>dmp_modeling_mode: raw_pool</code>
BA-gated DMP panel	<code>mc_dmp_fc</code>	<code>dmp_modeling_mode: featurecuts</code>
Enricher gene recurrence (PPI or Enrichr)	<code>mc_gene</code>	<code>stability_gene_recurrence_source: enricher</code> , <code>enricher.ppi_only</code>
Gene FeatureCuts on enricher/PPI genes	<code>mc_gene_fc</code>	<code>gene_modeling_mode: featurecuts</code>
Full DMP + gene FC (production)	<code>mc_dmp_gene_fc</code>	both axes featurecuts
Two-phase panel lock	<code>phase_a_dmp_stability</code> → <code>phase_b_gene_from_stable_dmps</code>	stable DMP panel then gene FC

```

methyl-workflow-run \
  --program workflow_engine/domain/checks/buffy_healthy_vs_pca/configs/buffy_mc_stability.prog
  --context-file workflow_engine/domain/profiles/mc_dmp_gene_fc.profile.json \
  --context '{"projectPath":"/work/projects/prostate-cancer/configs/project_Buffy_healthy_vs_P

```

Mapper CSV pattern follows modeling mode: `discovery` → `dmps-*-discovery.csv`; `featurecuts` → `dmps-*-selected.csv`. Downstream consumers should prefer **`dmps-*-selected.csv`**.

Deprecated names (`mc_dmp_discovery`, `mc_dmp_featurecuts`, `mc_gene_mapper`, `mc_gene_featurecuts`, `discovery_gene_featurecuts`, `dmp_panel_stability`, `gene_enricher_stability`, `buffy_mc_gene_fc`) alias to the canonical `mc_*` presets in `pipeline_profiles.py`.

7.5 Process model

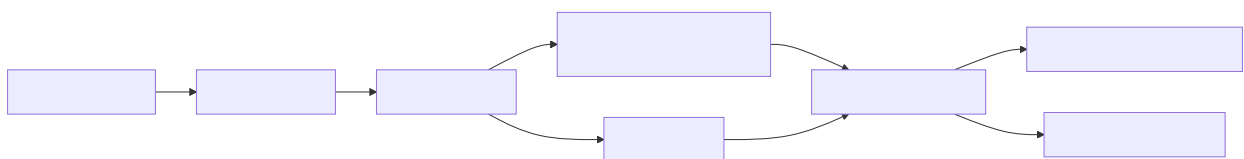


Figure 7.1: Stability stage process model

```

flowchart LR
    runs[run_0001..run_N] --> centroid[methyl-centroid] --> detector[methyl-detector]
    detector --> dmp[discovery + selected DMP exports]
    detector --> results[result JSON]
    dmp --> agg[run_stability_analysis]
    results --> agg
    agg --> stable[stable_dmps_production.csv]
    agg --> summary[stability_summary.json]

```

7.6 Expected outputs

- <mc_root>/stability/stable_dmps_production.csv
- <mc_root>/stability/stability_summary.json
- <mc_root>/all_metrics.csv
- <mc_root>/metrics_summary.json
- <mc_root>/baseline_manifest.json

Where <mc_root> is: <output_base>/<project_name>/monte_carlo_runs/

7.7 Success checks

- stable_dmps_production.csv exists and is non-empty.
- all_metrics.csv has successful rows near expected run count.
- stability_summary.json reports configured thresholds.
- if adaptive stop is enabled, stability_summary.json includes early_stopping diagnostics (triggered, checkpoint history, and stop iteration when applicable).

7.8 Do not do this

- Do not proceed to freeze when stable panel is missing or empty.
- Do not interpret blind outputs as stability quality evidence.

7.9 Recovery

- **Re-run workflow after MC complete (recommended):** re-run the same DomainProgram. Each action inside the MC FOREACH loop skips independently when its .action_results/manifest matches; post-MC steps (stability, readiness) skip the same way once they have run successfully once.

```
source .venv/bin/activate
methyl-workflow-run --program workflow_engine/domain/checks/.../buffy_mc_stability.program.json
--context-file workflow_engine/domain/profiles/your.profile.json
```

- **Stability-only (CLI, no workflow):** when MC outputs exist but stability never succeeded:

```
methyl-validation --project /work/.../configs/project.json --stability --skip-detection
```

- Resume interrupted MC iteration loop:

```
source .venv/bin/activate
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-C
```

- If panel is too small: lower `stability_dmp_freq` or increase `n_iterations`.
- If run quality is too low: tune detector and/or adjust `stability_min_balanced_accuracy`.

7.10 See also

- docs/theory/chapters/12-two-workflows.qmd
- docs/theory/chapters/15-model-creation-and-validation.qmd
- If many MC iterations are split across several hosts, use shared `output_base` and the distributed queue flow: [Distributed MethylValidation on shared storage](#) (Chapter 13).

8 Stage: Freeze

8.1 Purpose

Create frozen production artifacts using the stable panel and generate biological interpretation outputs (mapper/enricher, optional progression).

8.2 Entry command

```
source .venv/bin/activate
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-C
```

8.3 Prerequisites

- Stability stage completed.
- Stable panel available at:
 - configured profile `actionConfig.validation.freeze_stable_dmp_csv`, or
 - default `<mc_root>/stability/stable_dmps_production.csv`.

8.4 Process model



Figure 8.1: Freeze stage in pipeline lifecycle

8.5 Expected outputs

- `<mc_root>/production/project.json`
- `<mc_root>/production/production_summary.json`
- `<mc_root>/production/model_bundle/mapper_dmp_annotations.csv` (when mapped-family feature bundles are prepared)
- mapper outputs under `production/mapper/...`
- enricher outputs under `production/enricher/...`
- optional progression outputs under `production/progression/...`

8.6 Success checks

- `production/project.json` exists and includes fixed panel usage for detector.
- expected mapper and enricher files are present.
- stage-specific logs show successful return codes.
- for non-dmp observed-hybrid families, `production_summary.json` includes `mapper_annotation_cache` metadata.

8.7 Recommended post-freeze gate

Run biological readiness before model training:

```
source .venv/bin/activate
methyl-validation biological-readiness /work/projects/prostate-cancer/Healthy_vs_PCa1-4-CG
```

This chained gate verifies:

- enricher completeness (`verify-complete`),
- strict progression input completeness (`--strict-missing`),
- stability/freeze/progression readiness verdict.

To include advisory Grok commentary in the readiness section:

```
methyl-validation biological-readiness /work/projects/prostate-cancer/Healthy_vs_PCa1-4-CG --g
```

Without `--grok-review`, the chain keeps Grok disabled by default and still enforces deterministic checks.

8.8 Do not do this

- Do not run freeze before stability panel exists.
- Do not delete `production/project.json`; later model and predictor stages depend on it.

8.9 Recovery

- If freeze fails because stable panel missing, rerun stability or set `freeze_stable_dmp_csv`.
- If path mismatch occurs across machines, use `path_remap` or `--path-remap`.

8.10 See also

- `packages/methylvalidation/docs/USAGE.md`
- `docs/usage/index.qmd`

9 Stage: Model

9.1 Purpose

Train and select final production model artifacts after freeze and biological-readiness checks.

9.2 Common execution patterns

9.2.1 Pattern A: Direct production model build

```
source .venv/bin/activate
methyl-validation biological-readiness /work/projects/prostate-cancer/Healthy_vs_PCa1-4-CG
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG
```

9.2.2 Pattern B: Backend model-MC comparison then select best

```
source .venv/bin/activate
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG
--select-best-model --model-mc-all --selection-metric balanced_accuracy --selection-stat med
```

9.3 Backend behavior

- `ecdf`: `methyl-classifier` -> `methyl-predictor`
- `tabular_sklearn`: `in-process bundle` -> `train` -> `predict`
- `generative_hybrid`: `in-process bundle` -> `train` -> `predict`

For `ecdf` + `observed-hybrid` mapped families (`feature_mode=observed_hybrid`, `feature_family_set!=dmp_score`) the runtime path is aggregated ECDF OvR package `train/predict` and skips ECDF second-stage refinement.

Configured by profile `actionConfig.validation.backend_profiles` (plus optional CLI backend override).

9.4 Process model



Figure 9.1: Model stage in pipeline lifecycle

9.5 Required config keys

- `backend_profiles`
- backend-specific params under each profile (for example `feature_mode`, `ecdf_second_stage_enabled`, `tabular_methods`, `generative_latent_dim`)
- observed-hybrid family contract (when used):
 - `feature_family_set`
 - mapper annotation cache from freeze (`production/model_bundle/mapper_dmp_annotations.csv`)
- tabular controls:
 - `tabular_methods`
 - `tabular_method_selection_metric`
 - optional `tabular_max_dmps` (null/0 keeps all stable loci)
- generative controls:
 - `generative_latent_dim`, `generative_epochs`, and related keys
- optional gate:
 - `require_biological_review_for_model`
 - `biological_review_confirmed`

Legacy flat backend keys are rejected. Migrate older configs with:

```
methyl-validation-migrate-backend-config /path/to/project.json --in-place
```

9.6 Expected outputs

- `<mc_root>/production/classifiers/...`
- `<mc_root>/production/predictors/...`
- aggregated ECDF mode writes:
 - `<mc_root>/production/classifiers/ecdf_aggregated_ovr.pkl`
 - `<mc_root>/production/classifiers/ecdf_aggregated_ovr.meta.json`
- if model-MC:
 - `<mc_root>/model_mc/shared/run_XXXX/...`
 - `<mc_root>/model_mc/<backend>/all_metrics.csv`
 - `<mc_root>/model_mc/backend_ranking.csv`
 - `<mc_root>/production/selected_backend.json` (after selection)

When model-MC reuses primary split artifacts, shared/backend run roots may link centroid/detector directories instead of recomputing them.

9.7 Success checks

- final classifier/predictor artifacts exist in production root.
- selected backend metadata exists when using selection workflow.
- metrics summaries include expected selection metric fields.
- the last biological-readiness report is `go/go_with_risks` and reviewed by humans.

9.8 Do not do this

- Do not run model before freeze.
- Do not skip biological readiness review when production artifacts changed.
- Do not claim backend superiority without model-MC summaries.

9.9 See also

- `docs/theory/chapters/15-model-creation-and-validation.qmd`
- `packages/methylvalidation/docs/ROLLOUT.md`

10 Stage: Post-model Validation

10.1 Purpose

Measure descriptive performance distributions using frozen production artifacts without retraining.

10.2 Entry command

```
source .venv/bin/activate
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-C
```

Optional backend override:

```
source .venv/bin/activate
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-C
  --post-model-validation --post-model-backend ecdf
```

10.3 Prerequisites

- Freeze/model artifacts already built in production root.
- Valid production/project.json.

10.4 Process model



Figure 10.1: Post-model validation

10.5 Expected outputs

- <mc_root>/post_model_validation/run_XXXX/...
- <mc_root>/post_model_validation/all_metrics.csv
- <mc_root>/post_model_validation/metrics_summary.json
- <mc_root>/post_model_validation/metrics_distributions_plotly.html

10.6 Success checks

- run directories created under `post_model_validation`.
- metrics summary includes key metrics (`balanced_accuracy`, `macro_f1`, plus available calibration metrics).
- distribution plot generated.

10.7 Do not do this

- Do not treat post-model validation as model training.
- Do not compare candidate and baseline manually without fixed thresholds; use rollout comparison when promoting.

10.8 Recovery

- Resume interrupted post-model validation:

```
source .venv/bin/activate
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-C
--post-model-validation --resume
```

10.9 See also

- `packages/methylvalidation/docs/USAGE.md`
- `packages/methylvalidation/docs/ROLLOUT.md`

11 Stage: Blind Prediction

11.1 Purpose

Run inference on newly arrived unlabeled samples using frozen model artifacts.

11.2 Important boundary

Blind-only prediction is a standalone `methyl-predictor` operation. It is intentionally outside `methyl-validation` accuracy workflows.

11.3 Entry command (standalone)

```
source .venv/bin/activate
methyl-predictor --project /work/projects/prostate-cancer/Healthy_vs_PCa1-4-CG/monte_carlo_runs
```

11.4 Process model



Figure 11.1: Blind prediction stage

11.5 Required config pattern

Profile `actionConfig.predictor` should reference blind/test-blind inputs and model paths resolved from frozen production project.

11.6 Expected outputs

- blind prediction report JSON
- per-sample class probabilities and predicted subgroup labels

11.7 Success checks

- predictor completes without missing model-path errors.
- expected blind report file exists.
- per-sample prediction rows match input sample count.

11.8 Do not do this

- Do not use blind-only prediction results as labeled validation accuracy claims.
- Do not run blind-only predictor settings through `methyl-validation` MC paths.

11.9 Recovery

- If model path resolution fails, confirm `production/project.json` and classifier artifacts exist.
- If sample-id join fails, verify sample path naming and predictor input lists.

11.10 See also

- `packages/methylpredictor/docs/USAGE.md`
- `docs/usage/index.qmd`

Part III

Part III: Operations and Reliability

12 Artifacts and QA Checks

12.1 Purpose

Provide a single checklist for expected files at each stage and quick quality gates before proceeding.

12.2 Workflow action observability

Each workflow ACTION executed by `methyl-worker` submits a typed `output_json` to the database (validated against `schemas/tasks/*.json`). Outputs include timing (`duration_ms`), branch code (`result_code`), and paths to artifacts on shared storage.

On `/work`, look for:

- **Per-action manifests:** `{output_dir}/.action_results/{action_name}.{run_key}.json` — full typed result written by CLIs (e.g. `pipeline.dmp_select`, `pipeline.mapper`) or reconstructed by the worker collector.
- **Sample prep timeline:** `{sampleDir}/{sampleId}.sample_prep_log.jsonl` — append-only JSONL audit for download, align, QC, extract, archive steps.
- **Workflow action timeline:** `{logRoot}/action_run_log.jsonl` — append-only log for every workflow ACTION when a log root resolves (MC runs, project output, or sample dir). Records include timing, typed `inputs/outputs`, and idempotency fields.

Regenerate JSON Schema artifacts after model changes:

```
source .venv/bin/activate
methyl-export-task-schemas
methyl-export-task-schemas --check # CI drift gate
```

12.3 Artifact map

Stage	Required artifacts	Gate to proceed
Sample prep	<code>{sample_id}.json</code> , <code>alignment_qc/{sample_id}.json</code> , <code>{sample_id}.extraction_qc.json</code> , <code>{chr}-CG.h5</code>	<code>guardrails.overall_pass</code> <code>true</code> for alignment and extraction QC

Stage	Required artifacts	Gate to proceed
Stability	<code>stability/stable_dmps_production/stable_panel</code> , <code>stability_summary.json</code> , <code>all_metrics.csv</code>	Stable panel exists and is non-empty
Freeze	<code>production/project.json</code> , mapper/enricher outputs	Production project exists; fixed-panel detector run completed
Model	<code>production/classifiers/*</code> , <code>production/classifiers/training_metrics.json</code> (all model backends), <code>production/predictors/*</code>	Final classifier/predictor artifacts present
Post-model validation	<code>post_model_validation/all_metrics_summary.json</code> , plotly report	Distribution summaries generated
Blind prediction	blind prediction report + per-sample probabilities	All input blind samples receive predictions

12.4 Handoff integrity checks



Figure 12.1: Artifact handoffs

12.5 QA checklist per stage

12.5.1 Stability

- Iteration count in metrics roughly matches configured `n_iterations`.
- Stable panel size is consistent with selected thresholds.

12.5.2 Freeze

- `production/project.json` references fixed panel detector mode.
- Mapper/enricher outputs exist for each expected comparison group.

12.5.3 Model

- Backend used is recorded and consistent with config.
- If selection used, ranking artifacts exist and chosen backend is traceable.

12.5.4 Post-model validation

- Metrics summary includes required decision metrics.
- Distribution artifacts generated and readable.

12.5.5 Blind prediction

- Prediction outputs include all requested samples.
- Operational report does not include false accuracy claims.

12.6 Escalation triggers

- Empty stable panel.
- Missing production project.
- No model artifacts after model stage.
- Repeated run failures in same step across resumed runs.

12.7 See also

- `packages/methylvalidation/docs/USAGE.md`
- `packages/methylvalidation/docs/IMPLEMENTATION.md`

13 Troubleshooting and Recovery

13.1 Purpose

Standardize response for common stage failures and reduce ad-hoc debugging.

13.2 Failure routing

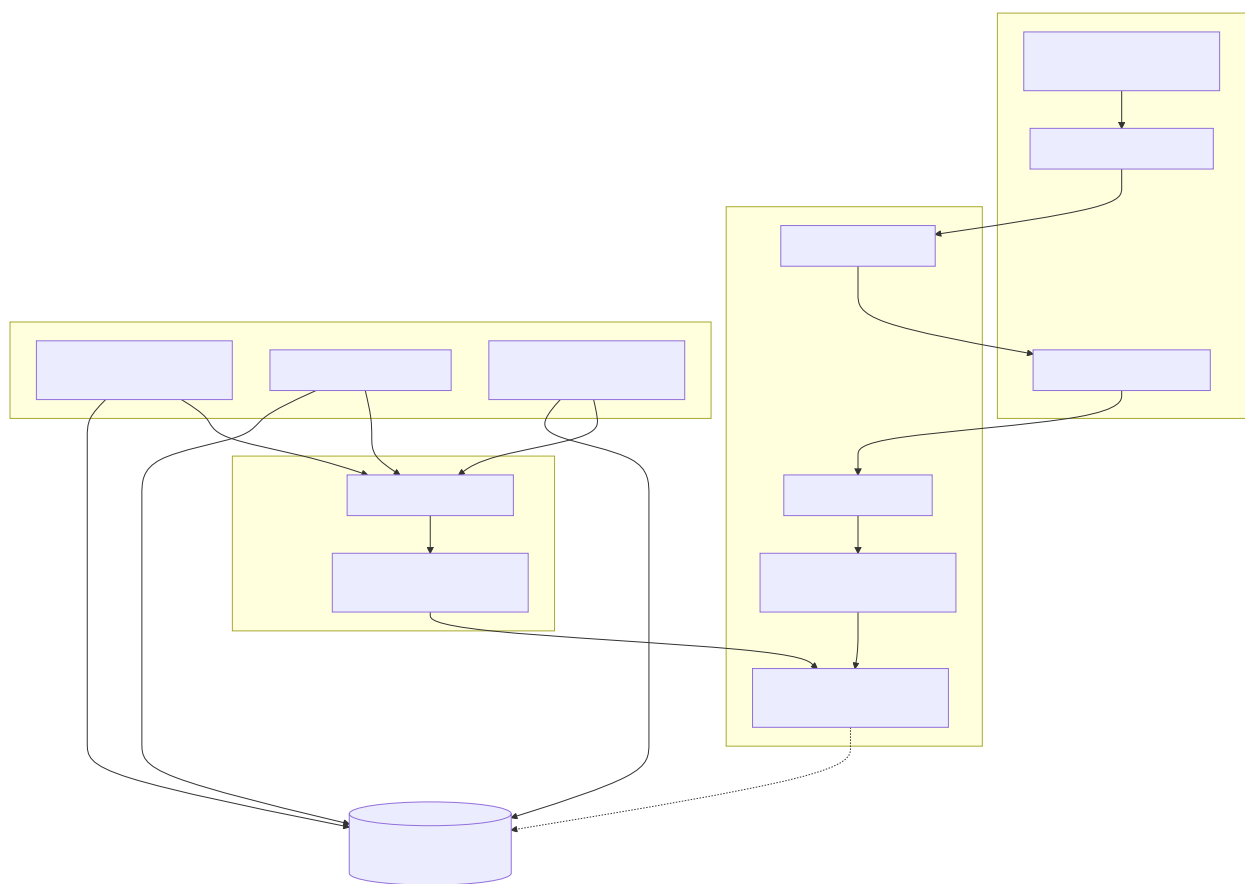


Figure 13.1: Failure routing

13.3 Common problems

Symptom	Likely cause	Recovery
command not found: methyl-validation	.venv inactive or install missing	Activate .venv, rerun scripts/install_all.sh
freeze fails for missing stable panel	stability not run or wrong path	Run --stability or set freeze_stable_dmp_csv
model fails due to biological review gate	require_biological_review_for_setmodel=true and not confirmed	biological_review_confirmed=true after review
path file-not-found across hosts	environment path differences	use path_remap/--path-remap OLD=NEW
blind predictor rejected in validation mode	blind settings passed to methyl-validation	run blind mode with standalone methyl-predictor

13.4 Safe recovery sequence

1. Validate environment and CLI availability.
2. Confirm previous-stage artifacts exist.
3. Re-run failed stage only.
4. Use resume mode where supported.
5. Re-check QA gates before advancing.

13.5 Resume examples

```
source .venv/bin/activate
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-C
methyl-validation --project /work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-C
```

13.6 Operational guardrails

- Keep original run directories for traceability.
- Prefer additive reruns over manual artifact edits.
- Record config overrides used in reruns.

13.7 See also

- docs/code_first_discovery_report.md
- docs/reference/config-parameter-matrix.md

14 Command Cookbook

14.1 Purpose

Provide copy-ready command sequences for the full solution lifecycle.

14.2 1) Full staged run (strict profile example)

```
source .venv/bin/activate
PROJECT="/work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json"

methyl-validation --project "$PROJECT" --stability
methyl-validation --project "$PROJECT" --freeze
methyl-validation --project "$PROJECT" --model
methyl-validation --project "$PROJECT" --post-model-validation
```

14.3 2) Backend comparison and selection

```
source .venv/bin/activate
PROJECT="/work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json"

methyl-validation --project "$PROJECT" --model-mc --model-mc-all
methyl-validation --project "$PROJECT" --select-best-model --model-mc-all \
  --selection-metric balanced_accuracy --selection-stat median
```

14.4 3) Rollout comparison report

```
source .venv/bin/activate
PROJECT="/work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json"
BASE="/work/projects/prostate-cancer/Healthy_vs_PCa1-4-CG/monte_carlo_runs/model_mc/ecdf/metric_01.json"
CAND="/work/projects/prostate-cancer/Healthy_vs_PCa1-4-CG/monte_carlo_runs/model_mc/tabular_sk_01.json"

methyl-validation --project "$PROJECT" --rollout-compare \
  --baseline-summary "$BASE" --candidate-summary "$CAND"
```

14.5 4) Standalone enricher run from mapper output

```
source .venv/bin/activate
methyl-enricher \
  --input /work/projects/prostate-cancer/Healthy_vs_PCa1-4-CG/mapper/all/pca_pca1/all-gene_name
  --modules \
  --library-preset cancer-extended \
  --outdir /work/projects/prostate-cancer/Healthy_vs_PCa1-4-CG/enricher/all/pca_pca1
```

14.6 5) Standalone blind prediction

```
source .venv/bin/activate
methyl-predictor --project /work/projects/prostate-cancer/Healthy_vs_PCa1-4-CG/monte_carlo_runs
```

14.7 6) Manual rendering commands (this manual)

```
quarto render docs/usage --to html
quarto render docs/usage --to pdf
```

14.8 7) Distributed queue (one task)

Full workflow: [Distributed MethylValidation on shared storage](#) (Chapter 13). One worker invocation:

```
source .venv/bin/activate
methyl-validation run-task --task /work/.../monte_carlo_runs/queue/tasks/run_0001.json
```

14.9 8) Hyperparameter search grid (example)

After `plan-runs` / full stability is understood, use the grid driver. Details: [Optional hyperparameter search](#) (Chapter 15).

```
source .venv/bin/activate
methyl-hyperparam-search \
  --config /work/experiments/my_mc_config.json \
  --work-dir /work/experiments/hp_run1 \
  --grid '{"stability_dmp_freq": [0.6, 0.7]}' \
  -- --stability
```

14.10 Execution notes

- Always activate `.venv` before `methyl-*` commands.
- Use absolute project paths in production scripts.
- Keep command logs with timestamp and git commit hash for audit.

15 Distributed MethylValidation on shared storage

15.1 Purpose

Run discovery Monte Carlo (and related queue-backed work) on **several machines** when a single host cannot finish `n_iterations` in acceptable wall time. Every worker must read and write the **same** `output_base` tree (commonly shared NFS under `/work/...`).

15.2 Prerequisites

- One `MonteCarloConfig` (dedicated `mc.json` or profile `actionConfig.validation` merged at plan time) with `output_base` on shared storage
- The same virtual environment and `methyl-*` CLIs on each worker
- R/W access to: `output_base/<project_name>/monte_carlo_runs/` and the `queue/` subtree created by planning

15.3 Layout (under `monte_carlo_runs/`)

Path	Role
<code>queue/mc_config.json</code>	Snapshotted config for workers
<code>queue/plan_runs.json</code>	Planned runs and task JSON paths
<code>queue/tasks/run_####.json</code>	One task = one <code>run-task</code> invocation
<code>queue/queue_manifest.jsonl</code> / <code>queue/commands.sh</code>	Optional export for a job scheduler
<code>run_####/queue_task_status.json</code>	Worker completion status
<code>run_####/queue_local_step_timings.json</code>	Merged by <code>aggregate-results</code> into <code>step_timings.csv</code>

Full path reference: `packages/methylvalidation/docs/DISTRIBUTED_QUEUE.md`.

15.4 Command sequence

1) **Plan** (one process; creates run dirs and task JSON, no heavy steps):

```
source .venv/bin/activate
methyl-validation plan-runs --config /work/experiments/my_mc_config.json --overwrite
# or: methyl-validation plan-runs --project /work/projects/prostate-cancer/configs/project_Hea
```

`--overwrite` only replaces `queue/tasks/` and regenerates the plan; it does **not** remove existing `run_####` output directories. To delete all `run_####` trees before planning, add `--wipe-runs` (irreversible). Without `--overwrite`, completed runs (see `queue_task_status.json` + per-run task under `queue/tasks/`) are not written over; you can add iterations and only the new `run_####` work is planned. `run-task` skips pipeline work for runs already marked completed unless you pass `--force`.

2) **Export** (optional; for a central queue broker):

```
methyl-validation export-queue --config /work/experiments/my_mc_config.json
```

3) **Workers** (one job per task; repeat until the queue is empty):

```
methyl-validation run-task --task /work/projects/prostate-cancer/Healthy_vs_PCa1-4-CG/monte_car
```

4) **Aggregate** (after successful tasks, or to refresh from a partial set):

```
methyl-validation aggregate-results --config /work/experiments/my_mc_config.json
# add --stability to run the same DMP/gene stability pass as monolithic methyl-validation --st
```

If adaptive stop settings are enabled for monolithic stability runs, remember queue workers do not coordinate global early termination on their own. Keep `stability_early_stop_enabled=false` for queue-first executions unless your scheduler supports coordinator-side cancellation logic.

Legacy: a single process may still run `methyl-validation --stability` without subcommands; behavior is unchanged.

15.5 Example paths (/work pattern)

With `output_base` pointing at shared storage, `plan-runs / export-queue` resolve `monte_carlo_runs` the same way as the monolithic CLI: `output_base / project_name / monte_carlo_runs /`.

15.6 Recovery

- Re-run a failed `run-task` for the same `--task` path.
- `queue_task_status.json` records failed vs completed.
- `aggregate-results` skips failed runs (when status files are present) and includes runs with computable per-run metrics.

15.7 See also

- `packages/methylvalidation/docs/DISTRIBUTED_QUEUE.md` (canonical detail)
- `docs/theory/chapters/15-model-creation-and-validation.qmd` (section *Distributed Monte Carlo on shared storage*)
- [Optional hyperparameter search](#) (Chapter 15; search trials can use the same shared storage)

16 Deployment and Distributed Workflow

16.1 Purpose

Deploy the **control plane** (database, REST gateway) and **GPU workers** that execute SamplePrep and validation workflows. This chapter consolidates the dual-backend story (Azure SQL and PostgreSQL) and links to detailed runbooks.

Deep dive: [docs/deployment/production_runbook.md](#)

Worker protocol: [workers/WORKER_PROTOCOL.md](#)

REST contract: [contracts/openapi.yaml](#)

16.2 Architecture

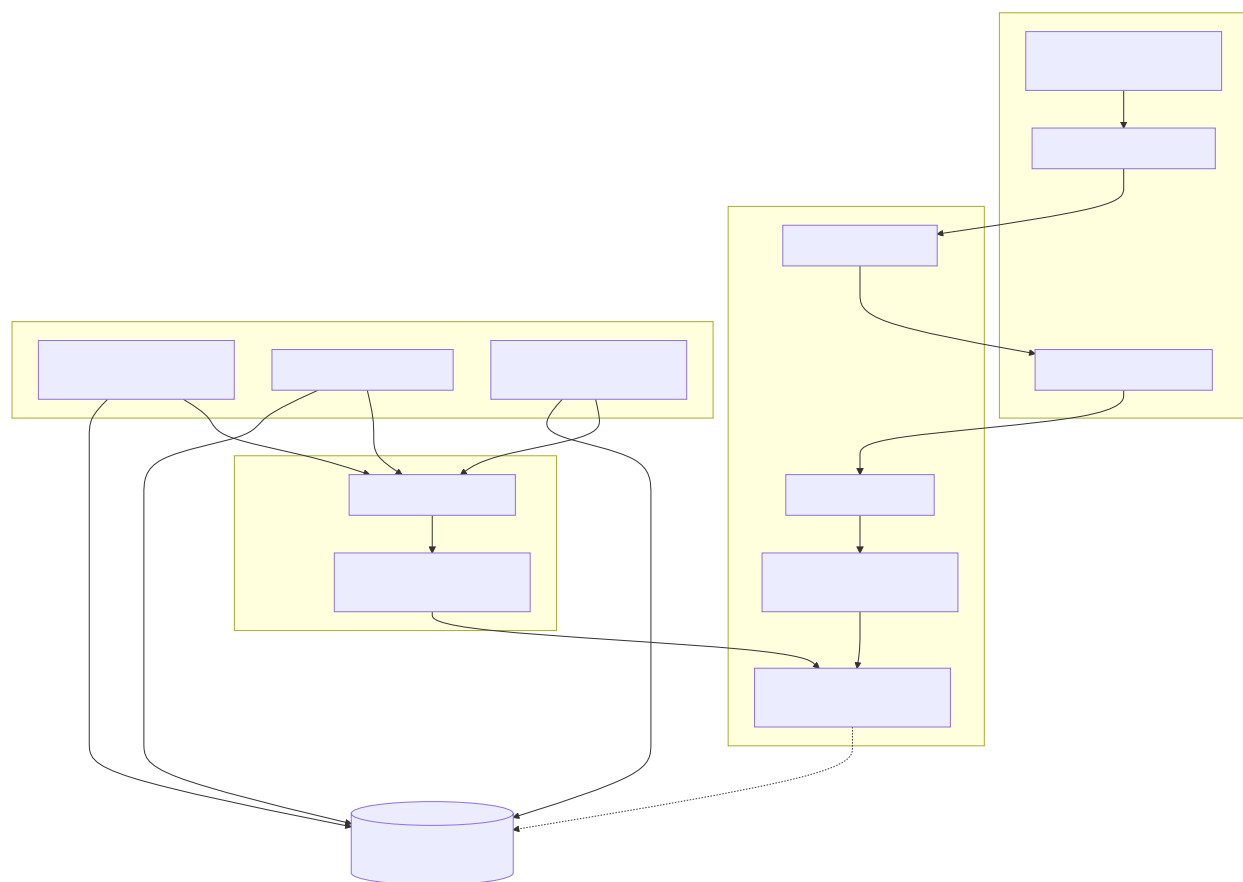


Figure 16.1: Distributed runtime

Component	Role
Database	Stores workflow definitions, instances, scope, task queue
Portal SQL (<code>portal.sp_*</code>)	Middle-tier orchestration; starts instances from study metadata
Gateway (<code>methy1-gateway</code>)	REST API for workers (poll/submit) and admin (deploy definitions)
Workers (<code>methy1-worker</code>)	Execute capabilities (Parabricks, <code>methy1-qc</code> , <code>detector</code> , ...)
/work storage	Shared FASTQs, BAMs, HDF5, MC outputs

The portal does **not** call the gateway for routine worker dispatch in production; workers poll the gateway directly. Operator/admin routes may require Entra ID JWT when `GATEWAY_REQUIRE_ENTRA=1`.

16.3 Greenfield checklist

Use this ordered checklist once per environment:

1. Mount shared storage at `/work/epimethyl` (or study-specific `/work/<disease>/`)
2. Deploy database schema (PostgreSQL or Azure SQL — see below)
3. Run portal SQL scripts (`portal_resource_profile.sql`, `portal_workflow_api.sql`, cluster security if used)
4. Seed action catalog: `methy1-export-task-schemas && methy1-export-action-catalog && python workflow_engine/sql_mssql/seed_action_catalog.py`
5. Deploy workflow definitions: `bash scripts/deploy_workflow_definitions.sh`
6. Install and configure gateway (`deploy/env/gateway.postgres.env.example` or `gateway.mssql.env.example` → `/work/epimethyl/env/gateway.env`)
7. Enable `methy1-gateway` systemd + nginx TLS (see [production_runbook.md](#))
8. Provision GPU workers: [worker_provision.md](#), [gpu_worker_runbook.md](#)
9. Register workers: `scripts/register_worker.sh`
10. Smoke test: `scripts/smoke_sample_prep.sh`, `scripts/smoke_study_lifecycle.sh`

16.4 Database deploy (dual-backend)

Both backends implement the same **wf** schema contract. Choose one primary backend per environment; CI and local dev often use PostgreSQL.

16.4.1 PostgreSQL (recommended for new deployments)

Step	Command / doc
Schema scripts	workflow_engine/sql_pg/README.md
Automated deploy	<code>./workflow_engine/sql_pg/deploy_azure.sh</code> (core engine + portal DDL)
Cluster IP binding	Optional: <code>wf_cluster_security_columns.sql</code> (run manually when using IP-bound clusters)
FOREACH support	Included in <code>08_foreach_support.sql</code>

```
export PGHOST=your-server.postgres.database.azure.com
export PGDATABASE=postgres
export PGUSER=dba
export PGPASSWORD='...'
export PGSSLMODE=require
./workflow_engine/sql_pg/deploy_azure.sh

# Optional: cluster IP binding columns (when using wf.cluster security features)
psql -f workflow_engine/sql_pg/wf_cluster_security_columns.sql
```

16.4.2 Azure SQL (legacy / phase-1)

Step	Command / doc
Script order	workflow_engine/README.md
Bundled schema	<code>workflow_engine/sql_mssql/MethylPipeline.sql</code>
FOREACH	<code>wf_sql_foreach_support.sql</code> (both backends)

```
# Example: sqlcmd against Azure SQL with scripts 1-18 from workflow_engine/README.md
```

Note: `deploy_azure.sh` applies core engine scripts plus portal DDL (`portal_resource_profile.sql`, `portal_workflow_api.sql`, `wf_drop_platform_sample_storage.sql`). Run `wf_cluster_security_columns.sql` separately when enabling cluster IP binding.

16.5 Action catalog and workflow definitions

After schema deploy:

```
source .venv/bin/activate
methyl-export-task-schemas
methyl-export-action-catalog
python workflow_engine/sql_mssql/seed_action_catalog.py

# Deploy SamplePrep + StudyValidation DomainPrograms to gateway:
export GATEWAY_URL=https://your-gateway.example.com
```

```
export GATEWAY_ADMIN_BEARER_TOKEN='...' # when Entra required
bash scripts/deploy_workflow_definitions.sh
```

Output: `workflow_versions.json` with `workflow_version_id` values for API calls.

Canonical programs (repo):

- SamplePrep: [workflow_engine/domain/fixtures/sample_prep.program.json](#)
- Study validation: check-specific `*.program.json` under `workflow_engine/domain/checks/`

Authoring language: [docs/reference/domain-program-language.md](#)

16.6 Gateway configuration

Install from repo root:

```
source .venv/bin/activate
pip install -e workflow_engine/
```

Copy and edit an example env file:

Backend	Template
PostgreSQL	deploy/env/gateway.postgres.env.example
Azure SQL	deploy/env/gateway.mssql.env.example

Install systemd unit [deploy/systemd/methyl-gateway.service](#) and nginx config [deploy/nginx/methyl-gateway.conf](#).

Key variables (see `workflow_engine/rest/connection.py`):

- `BACKEND_DB` — `postgres` or `mssql`
- `POSTGRES_*` or `AZURE_SQL_*` — connection targets
- `WF_USE_MANAGED_IDENTITY` — Azure Entra token auth for DB
- `GATEWAY_REQUIRE_ENTRA` — require JWT on admin/operator routes
- `GATEWAY_ENTRA_AUDIENCE`, `AZURE_TENANT_ID` — Entra validation

16.7 Worker configuration

Each GPU worker runs one or more `methyl-worker` systemd units (see [deploy/systemd/methyl-worker@.service](#)).

Variable	Purpose
<code>WORKER_ID</code>	Unique worker name

Variable	Purpose
WORKER_TOKEN	Auth token from registration
WORKER_CAPABILITY	e.g. <code>parabricks.fq2bam</code> , <code>methy1-qc</code> , <code>pipeline.detector</code>
WORKER_API_BASE	Gateway URL (https://gateway.example.com)

Registration and provisioning:

- [docs/deployment/worker_provision.md](#)
- [docs/deployment/worker_node.md](#)
- `scripts/register_worker.sh`, `scripts/provision_worker_node.sh`

Worker poll/submit contract: [workers/WORKER_PROTOCOL.md](#)

16.8 End-to-end operator flow

16.8.1 1. SamplePrepPipeline

```
POST /v1/studies/sample-prep/start
{
  "projectPath": "/work/epimethyl/data/project_Example.json",
  "workflow_version_id": <from workflow_versions.json>,
  "fastqStorage": { "type": "s3", "bucket": "...", "region": "us-west-2" },
  "sampleCsvs": ["/work/.../samples.csv"]
}
```

See [Sample Prep and Quality Control](#) for QC gates.

16.8.2 2. Study validation / MC

After SamplePrep **COMPLETED**, start validation via portal SQL or:

```
methy1-workflow-run \
  --program workflow_engine/domain/checks/.../configs/your_mc.program.json \
  --context-file workflow_engine/domain/profiles/your.profile.json \
  --context '{"projectPath":"/work/.../configs/project.json}"'
```

Or staged CLI: [Stage: Stability](#).

16.8.3 3. Local workflow (no gateway)

For development without database:

```
methy1-workflow-run --program path/to/program.json \  
  --context-file profile.json \  
  --context '{"projectPath":"/work/.../project.json"}' \  
  --stub-external
```

16.9 Related documentation

- Production runbook: [docs/deployment/production_runbook.md](#)
- Release layout: [docs/deployment/production_release.md](#)
- Portal lifecycle: [workflow_engine/docs/portal_study_lifecycle.md](#)
- SamplePrep operator guide: [workflow_engine/sql_mssql/SamplePrepFlow.md](#)
- Documentation audit: [docs/DOCUMENTATION_AUDIT.md](#)

17 Optional hyperparameter search

17.1 When to use

After a baseline `methyl-validation` path is understood, use a controlled **outer loop** to compare candidate settings—typically **Tier A** knobs in profile `actionConfig.validation` (e.g. `stability_dmp_freq`, `n_iterations` as budget, `stability_min_balanced_accuracy`). Tuning detector and model backends (Tier B/C) is a separate, often **nested** exercise; see the theory chapter on multi-stage search tradeoffs in `docs/theory/chapters/15-model-creation-and-validation.qmd` (*Optional pipeline hyperparameter search*).

17.2 What it is

- A **read-only** scalar **objective** (J) is computed from artifacts under `monte_carlo_runs/` (mainly `metrics_summary.json`, optional `stability/stability_summary.json`).
- The `methyl-hyperparam-search` entry point is a small **grid** driver: for each grid point it writes `work_dir/trial_NNNN/mc_config.json` with a merged `MonteCarloConfig` and a distinct `output_base`, runs `methyl-validation` (e.g. with `--stability`), then scores the run.
- It does **not** use `sklearn.model_selection` CV: each evaluation is a subprocess and file read.

Formula and weights: `packages/methylvalidation/docs/HYPERPARAMETER_SEARCH.md` (full (J), `ObjectiveWeights`, constraint semantics).

17.3 Command example

Use a dedicated MC config and a work directory on shared storage if many trials:

```
source .venv/bin/activate
methyl-hyperparam-search \
  --config /work/experiments/my_mc_config.json \
  --work-dir /work/experiments/hp_run1 \
  --grid '{"stability_dmp_freq": [0.6, 0.7], "stability_min_balanced_accuracy": [null, 0.5]}' \
  -- \
  --stability
```

- JSON `null` in `--grid` omits that field so the base config value is kept.
- Results: `work-dir/search_summary.json` (per-trial rows and best feasible trial, if any).
- `--weights-json` can point to a file for `ObjectiveWeights`.

- `--baseline-summary /path/to/metrics_summary.json` enforces **rollout-style** feasibility vs a baseline (same idea as `methyl-validation --rollout-compare`); see `packages/methylvalidation/docs/ROLLOUT.md` and `packages/methylvalidation/docs/HYPERPARAMETER`
- `--dry-run` only writes trial configs and paths; no pipeline subprocesses.

17.4 Pitfalls

- **Discovery vs calibration:** settings that optimize stability panel size may not optimize NLL, Brier, or ECE; prefer a **two-stage** strategy or tight constraints, not a single ad hoc weight mix.
- **n_iterations:** treat primarily as **compute budget** to stabilize medians, not a dimension to grid-search widely without cost awareness.

17.5 See also

- `packages/methylvalidation/docs/HYPERPARAMETER_SEARCH.md`
- `packages/methylvalidation/docs/ROLLOUT.md` (promotion / dual-run thinking)
- `docs/theory/chapters/15-model-creation-and-validation.qmd` (*Optional pipeline hyperparameter search*)
- [Distributed MethylValidation on shared storage](#) (Chapter 13; parallel execution of a single config's iterations)

Part IV

Part IV: Worked Tutorials

18 Tutorial: Healthy vs. Cancer (single group or increasing stages)

18.1 What you will build

A comparison study that contrasts **one healthy control group** against either:

- **one cancer group** (binary: healthy vs. cancer), or
- **several cancer groups in increasing stages** (healthy vs. stage 1 vs. stage 2 ...).

The pipeline treats both the same way: one control pool is compared against **each** disease group. Multi-stage is just “more disease groups,” so this tutorial covers both at once.

This is a short, end-to-end walkthrough. For field-by-field detail see [Project config and layout](#); for the stability stage see [Stage: Stability](#).

18.2 Mental model (four layers)

MethylPipeline is disease-agnostic. You describe **your study** (who is healthy, who is cancer, where the samples are) in a **study manifest**; the **science parameters** (iterations, thresholds, caps) come from a **pipeline profile** shipped in the repo. You never hard-code study facts in a profile, and you never put tool parameters in a manifest.

Layer	Artifact	You edit it?
Study manifest	<code>/work/projects/<disease>/configs/project-manifest.json</code>	Yes, this is your study
Cohort lists	<code>/work/projects/<disease>/data/*.csv</code>	Yes, one CSV per group
Pipeline profile	<code>workflow_engine/domain/profiles/*pipeline.json</code>	No, pick one
DomainProgram	<code>workflow_engine/domain/checks/**/domainprogram.json</code>	No, pick one

18.3 Step 1 — Lay out cohort CSVs

Put one sample list per group under `/work/projects/<disease>/data/`. Each CSV has a **sample** header and one sample folder name per row. Names resolve against `samples_base_path` (typically `/work/samples/<name>`).

`/work/projects/prostate-cancer/data/PCaH.csv` (healthy):

```
sample
HBCST-052125-87293
HBCST-061825-52197
HBCST-051425-74294
```

Create one more for the binary case (`pca_b.csv`), or one per stage for the multi-stage case (`PCa1.csv`, `PCa2.csv`, ...). A row may also be an absolute path or a direct sample directory; bare names use `samples_base_path`.

18.4 Step 2 — Write the study manifest

18.4.1 Option A — Binary (healthy vs. one cancer group)

`/work/projects/prostate-cancer/configs/project_Buffy_healthy_vs_PCa.json`:

```
{
  "project_name": "Buffy_healthy_vs_PCa",
  "output_base": "/work/projects/prostate-cancer",
  "samples_base_path": "/work/samples",
  "controls": {
    "label": "healthy",
    "groups": [
      { "label": "all", "sample_paths": ["/work/projects/prostate-cancer/data/healthy_b.csv"] }
    ]
  },
  "diseases": {
    "label": "cancer",
    "groups": [
      { "label": "PCa", "sample_paths": ["/work/projects/prostate-cancer/data/pca_b.csv"] }
    ]
  },
  "comparisons": "control_vs_each_disease",
  "chromosomes": ["1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "11", "12", "13", "14", "15", "16", "17", "18", "19", "20", "X", "Y"],
  "contexts": ["CG"]
}
```

18.4.2 Option B — Increasing stages (healthy vs. stage 1 ... stage N)

To add stages, nest a `stages[]` array under a single disease parent. Each stage is its own group with its own CSV. **The parent must not carry `sample_paths` when it uses `stages`.** Stages expand to resolved labels of the form `{parent}_{stage}` — here `PCa_PCa1` ... `PCa_PCa5`.

```

{
  "project_name": "Healthy_vs_PCa1-5-CG",
  "output_base": "/work/projects/prostate-cancer",
  "samples_base_path": "/work/samples",
  "controls": {
    "label": "healthy",
    "groups": [
      { "label": "all", "sample_paths": ["/work/projects/prostate-cancer/data/PCaH.csv"] }
    ]
  },
  "diseases": {
    "label": "cancer",
    "groups": [
      {
        "label": "PCa",
        "stages": [
          { "label": "PCa1", "description": "Gleason Score 3+3", "sample_paths": ["/work/p",
          { "label": "PCa2", "description": "Gleason Score 3+4", "sample_paths": ["/work/p",
          { "label": "PCa3", "description": "Gleason Score 4+3", "sample_paths": ["/work/p",
          { "label": "PCa4", "description": "Gleason Score 4+4", "sample_paths": ["/work/p",
          { "label": "PCa5", "description": "Gleason Score 4+5/5+4", "sample_paths": ["/work/p",
        ]
      }
    ]
  },
  "comparisons": "control_vs_each_disease",
  "progression_order": "explicit",
  "progression_labels": ["PCa_PCa1", "PCa_PCa2", "PCa_PCa3", "PCa_PCa4", "PCa_PCa5"],
  "chromosomes": ["1","2","3","4","5","6","7","8","9","10","11","12","13","14","15","16","17",
  "contexts": ["CG"]
}

```

A complete, annotated version of this staged manifest lives at `workflow_engine/domain/checks/pca1_5_cg/config/5-CG.json`.

18.5 Step 3 — Understand what comparisons does

`comparisons` decides which group pairs get analyzed. For a healthy-vs-stages study you almost always want `"control_vs_each_disease"`.

Value	Meaning	Use for
<code>"control_vs_each_disease"</code>	First control group vs. each disease group/stage	Healthy vs. stage 1, stage 2, stage 3 ... (this tutorial)
<code>"all_pairs"</code>	Every control group × every disease group	Multiple control strata

Value	Meaning	Use for
[{ "control_group": "all", "disease_group": "PCa_PCa1" }, ...]	Hand-picked pairs (using resolved labels)	Custom subsets

Important: "control_vs_each_disease" compares **healthy against each stage** — it does **not** run stage-vs-stage contrasts. If you omit **comparisons** entirely, the pipeline defaults to **control_vs_each_disease** for a single control group. To model the ordered stage narrative (early → advanced) after per-comparison detection, set **progression_order** / **progression_labels** as in Option B.

18.6 Step 4 — Pick a profile

The profile supplies the science knobs (iterations, thresholds, feature caps). Pick by what you want out of the run:

Goal	pipelineProfile
Standard binary Monte-Carlo stability (DMP + gene FeatureCuts)	mc_dmp_gene_fc
Multi-stage, one-vs-rest Monte-Carlo stability	staged_ovr_mc
Multi-stage end-to-end (stability + freeze + model)	staged_full_lifecycle
Progression / interpretation only	staged_progression_interpretation

All profiles live in `workflow_engine/domain/profiles/`. See the profile/mode table in [Stage: Stability](#).

18.7 Step 5 — Run it

Always activate the repo virtualenv first. You pass three things: a **program** (the workflow topology), a **profile** (via `--context-file`), and a **context** JSON that points at your manifest and names the profile.

Binary (healthy vs. one cancer group):

```
source .venv/bin/activate
methyl-workflow-run \
  --program workflow_engine/domain/checks/buffy_healthy_vs_pca/configs/buffy_mc_stability.prog
  --context-file workflow_engine/domain/profiles/mc_dmp_gene_fc.profile.json \
  --context '{"projectPath":"/work/projects/prostate-cancer/configs/project_Buffy_healthy_vs_P
  --parallel-workers 1
```

Multi-stage (healthy vs. PCa1...PCa5):

```
source .venv/bin/activate
methyl-workflow-run \
  --program workflow_engine/domain/checks/pca1_5_cg/configs/pca1_5_mc_stability.program.json \
  --context-file workflow_engine/domain/profiles/staged_ovr_mc.profile.json \
  --context '{"projectPath":"/work/projects/prostate-cancer/configs/project_Healthy_vs_PCa1-5-
  --parallel-workers 1
```

Before a full run, you can validate the manifest and compile the program with the check harness:

```
source .venv/bin/activate
python workflow_engine/domain/checks/pca1_5_cg/check_pipeline.py \
  --program workflow_engine/domain/checks/pca1_5_cg/configs/pca1_5_mc_stability_smoke.program.
```

In production, point `--program` and `--context-file` at the promoted runtime bundle instead of the repo, e.g. `/work/epimethyl/current/runtime-bundle/domain/...` (see [Deployment and distributed workflow](#)).

18.8 Step 6 — Find the outputs

Results land under `output_base/project_name/`. Each comparison gets its own detection folder keyed by control and disease group:

```
/work/projects/prostate-cancer/Healthy_vs_PCa1-5-CG/
  detections/all/PCa_PCa1/      # healthy vs. stage 1
  detections/all/PCa_PCa2/      # healthy vs. stage 2
  ...
  monte_carlo_runs/             # MC iterations, stability summaries
```

For a binary study the single comparison appears under `detections/all/PCa/`. Monte-Carlo stability writes recurrence panels and `stability_summary.json` under `monte_carlo_runs/`; see [Artifacts and QA checks](#) for what each artifact means and the handoff checks before the freeze stage.

18.9 Recap

1. One CSV per group under `data/`.
2. One manifest: `controls + diseases` (nest `stages[]` for increasing stages), `comparisons:` `"control_vs_each_disease"`.
3. Pick a profile (`mc_dmp_gene_fc` binary dual-axis, `mc_dmp / mc_dmp_fc / mc_gene / mc_gene_fc` for research axes, `staged_ovr_mc` multi-stage).
4. `methy1-workflow-run --program ... --context-file ...profile.json --context '{"projectPath":..., "pipelineProfile":...}'`.
5. Read results per comparison under `<project_name>/detections/<control>/<disease>/`.

18.10 See also

- [Project config and layout](#) — every manifest field
- [Stage: Stability](#) — profiles, modeling modes, required `actionConfig` keys
- [DomainProgram language](#) — program vs. project vs. profile vs. context
- `packages/methylutils/docs/COHORT_TREE.md` — stages and comparison-shorthand semantics
- [Command cookbook](#) — copy-ready command sequences